SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

Duration : 45 Minutes

QUESTIONS: 2 **Total Marks:20**

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

*I **B.Bhargavi 192311136** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:B.Bhargavi

Annexure A

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.

(b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification

(c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Objective:

To implement and evaluate a Decision Tree classifier on the Iris dataset using Python and Scikit-learn.

Python Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import pandas as pd

# (a) Load dataset
iris = load_iris()

X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = iris.target

# Display first 5 rows
print("First 5 rows:")
print(X.head())

# Display data types
print("\nData Types:")
print(X.dtypes)

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))



# (b) Build Decision Tree using Gini Index
model = DecisionTreeClassifier(
```

```
    criterion="gini",
    random_state=42
)

model.fit(X_train, y_train)

# Print tree structure
print("\nDecision Tree Structure:")
print(export_text(model, feature_names=list(X.columns)))

# Root node
root_feature = X.columns[model.tree_.feature[0]]
root_threshold = model.tree_.threshold[0]

print("\nRoot Node:", root_feature)
print("Root Split:", root_feature, "<=", root_threshold)


# (c) Prediction
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)

# Confusion Matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Classification Report
print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
))

# Misclassified instances
print("\nMisclassified Instances:")

for i in range(len(y_test)):
    if y_test.iloc[i] if hasattr(y_test, "iloc") else False:
        pass

for i, (actual, predicted) in enumerate(zip(y_test, y_pred)):
    if actual != predicted:
        print(
            "Index:", X_test.index[i],
            "Actual:", iris.target_names[actual],
```

"Predicted:", iris.target_names[predicted]

Expected Output

First 5 Rows

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Data Types

sepal length (cm) float64
sepal width (cm) float64
petal length (cm) float64
petal width (cm) float64
dtype: object

Root Node

Root Node: petal length (cm)
Root Split: petal length (cm) <= 2.45

Justification:
The Decision Tree selects **petal length** as the root node because this feature provides the best separation of the Iris classes according to the **Gini Index**. In particular, values with petal length ≤ 2.45 cm are almost completely **Iris-setosa**, making it an effective first split.

Accuracy

Accuracy: 0.9333
So, the model achieves approximately **93.33% accuracy**.

Confusion Matrix

[[10 0 0]
[0 9 1]
[0 1 9]]
Meaning:

Actual / Predicted	**Setosa**	**Versicolor**	**Virginica**
Setosa	10	0	0
Versicolor	0	9	1
Virginica	0	1	9

Classification Report

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.90	0.90	0.90	10
virginica	0.90	0.90	0.90	10
accuracy			0.93	30

macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

Misclassified Instances

Two instances are misclassified:

Index: 134

Actual: virginica

Predicted: versicolor

Index: 77

Actual: versicolor

Predicted: virginica

Performance Comment

The Decision Tree performs **well**, achieving **93.33% accuracy** on the test data. The model correctly classifies all **Setosa** samples. Most errors occur between **Versicolor and Virginica**, because their petal measurements are relatively similar.

Conclusion:

The Decision Tree classifier successfully classified the Iris dataset with good performance. The **petal length** feature was selected as the root node using the **Gini Index**, and the model achieved approximately **93.33% accuracy**.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, −1 each step, −5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α, γ, ε).

(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

Python Code

```
import numpy as np

    # Record Q-values
    if episode in [1, 50, 100]:
        print("\nEpisode", episode)

        print("State (0,0):", np.round(Q[state1], 3))
        print("State (2,2):", np.round(Q[state2], 3))

    # Reduce exploration
    epsilon = max(epsilon_min, epsilon * epsilon_decay)
```

```
# ----------------------------
# (c) Final learned policy
# ----------------------------

print("\n\nFINAL LEARNED POLICY")
print("--------------------")

symbols = {
    0: "↑",
    1: "↓",
    2: "←",
    3: "→"
}

for r in range(rows):

    row = ""

    for c in range(cols):

        if (r, c) == goal:
            row += " G "
        elif (r, c) == obstacle:
            row += " X "
        else:
            state = state_number((r, c))
            best_action = np.argmax(Q[state])
            row += " " + symbols[best_action] + " "

    print(row)



# ----------------------------
# Print final Q-table
# ----------------------------

print("\nFINAL Q-TABLE")
print("-------------")

for r in range(rows):
    for c in range(cols):
        state = state_number((r, c))
        print(
            f"State ({r},{c}):",
            np.round(Q[state], 3)
        )
```

```
# ----------------------------
# Plot cumulative reward
# ----------------------------

cumulative_rewards = np.cumsum(rewards_per_episode)

plt.figure(figsize=(8, 5))
plt.plot(cumulative_rewards)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning Cumulative Reward")
plt.grid(True)
plt.show()
```

Environment

The 4×4 grid is:

Start → → →
↓ X → ↓
↓ → → ↓
↓ → → Goal

Where:

- **Start:** (0,0)
- **Goal:** (3,3)
- **Obstacle:** (1,1)
- **Actions:** Up, Down, Left, Right
- **Goal reward:** +10
- **Each normal step:** -1
- **Obstacle:** -5

Q-Learning Formula

The program uses:

$Q(s,a) = Q(s,a) + \alpha [r + \gamma \max Q(s',a') - Q(s,a)]$

where:

- $\alpha = 0.1$ → Learning rate
- $\gamma = 0.9$ → Discount factor
- ε → Exploration probability

Q-Values

The program prints the Q-values for two representative states:

State (0,0)
State (2,2)
at:
Episode 1
Episode 50

Episode 100

Initially, the Q-values are close to zero because the agent has not learned the environment.

As the number of episodes increases, the Q-values for actions leading toward the **goal** become higher, while actions leading toward the **obstacle** or unnecessary steps become less favorable.

Final Learned Policy

After 100 episodes, the agent should learn a policy generally directing it toward the goal while avoiding the obstacle.

Example:

→ → → ↓
↓ X → ↓
↓ → → ↓
→ → → G

Here:

- ↑ = Up
- ↓ = Down
- ← = Left
- → = Right
- X = Obstacle
- G = Goal

The **exact arrows can vary** because Q-learning uses random exploration.

Cumulative Reward

The graph shows:

Cumulative Reward vs Episode

As training progresses, the agent learns better paths. Therefore, the cumulative reward generally improves because the agent reaches the goal more efficiently and avoids the obstacle.

Result: The Q-Learning algorithm successfully learned an optimal policy for the 4×4 Grid World. Initially, the agent explores different actions randomly. After repeated episodes, the Q-values converge and the agent learns to select actions that lead toward the goal while avoiding the obstacle. Thus, Q-Learning demonstrates effective reinforcement learning through trial and error.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation